interp_rbf.h

```
struct RBF_multiquadric : RBF_fn {
```
Instantiate this and send to RBF_interp to get multiquadric interpolation.
```
    Doub r02;
    RBF_multiquadric(Doub scale=1.) : r02(SQR(scale)) {}
```
    Constructor argument is the scale factor. See text.
```
    Doub rbf(Doub r) { return sqrt(SQR(r)+r02); }
};

struct RBF_thinplate : RBF_fn {
```
Same as above, but for thin-plate spline.
```
    Doub r0;
    RBF_thinplate(Doub scale=1.) : r0(scale) {}
    Doub rbf(Doub r) { return r <= 0. ? 0. : SQR(r)*log(r/r0); }
};

struct RBF_gauss : RBF_fn {
```
Same as above, but for Gaussian.
```
    Doub r0;
    RBF_gauss(Doub scale=1.) : r0(scale) {}
    Doub rbf(Doub r) { return exp(-0.5*SQR(r/r0)); }
};

struct RBF_inversemultiquadric : RBF_fn {
```
Same as above, but for inverse multiquadric.
```
    Doub r02;
    RBF_inversemultiquadric(Doub scale=1.) : r02(SQR(scale)) {}
    Doub rbf(Doub r) { return 1./sqrt(SQR(r)+r02); }
};
```

Typical use of the objects in this section should look something like this:

```
Int npts=...,ndim=...;
Doub r0=...;
MatDoub pts(npts,ndim);
VecDoub y(npts);
...
RBF_multiquadric multiquadric(r0);
RBF_interp myfunc(pts,y,multiquadric,0);
```

followed by any number of interpolation calls,

```
VecDoub pt(ndim);
Doub val;
...
val = myfunc.interp(pt);
```

3.7.3 Shepard Interpolation

An interesting special case of normalized radial basis function interpolation (equations 3.7.3 and 3.7.4) occurs if the function $\phi(r)$ goes to infinity as $r \to 0$, and is finite (e.g., decreasing) for $r > 0$. In that case it is easy to see that the weights w_i are just equal to the respective function values y_i, and the interpolation formula is simply

$$y(\mathbf{x}) = \frac{\sum_{i=0}^{N-1} y_i \phi(|\mathbf{x} - \mathbf{x}_i|)}{\sum_{i=0}^{N-1} \phi(|\mathbf{x} - \mathbf{x}_i|)} \tag{3.7.9}$$

(with appropriate provision for the limiting case where $\mathbf{x}$ is equal to one of the $\mathbf{x}_i$'s). Note that no solution of linear equations is required. The one-time work is negligible, while the work for each interpolation is $O(N)$, tolerable even for very large N.

Shepard proposed the simple power-law function

$$\phi(r) = r^{-p} \tag{3.7.10}$$

with (typically) $1 < p \lesssim 3$, as well as some more complicated functions with different exponents in an inner and outer region (see [8]). You can see that what is going on is basically interpolation by a nearness-weighted average, with nearby points contributing more strongly than distant ones.

Shepard interpolation is rarely as accurate as the well-tuned application of one of the other radial basis functions, above. On the other hand, it is simple, fast, and often just the thing for quick and dirty applications. It, and variants, are thus widely used.

An implementing object is

interp_rbf.h

```
struct Shep_interp {
```

Object for Shepard interpolation using `n` points in `dim` dimensions. Call constructor once, then `interp` as many times as desired.

```
    Int dim, n;
    const MatDoub &pts;
    const VecDoub &vals;
    Doub pneg;

    Shep_interp(MatDoub_I &ptss, VecDoub_I &valss, Doub p=2.)
    : dim(ptss.ncols()), n(ptss.nrows()) , pts(ptss),
    vals(valss), pneg(-p) {}
```

Constructor. The `n` × `dim` matrix `ptss` inputs the data points, the vector `valss` the function values. Set `p` to the desired exponent. The default value is typical.

```
    Doub interp(VecDoub_I &pt) {
```

Return the interpolated function value at a `dim`-dimensional point `pt`.

```
        Doub r, w, sum=0., sumw=0.;
        if (pt.size() != dim) throw("RBF_interp bad pt size");
        for (Int i=0;i<n;i++) {
            if ((r=rad(&pt[0],&pts[i][0])) == 0.) return vals[i];
            sum += (w = pow(r,pneg));
            sumw += w*vals[i];
        }
        return sumw/sum;
    }

    Doub rad(const Doub *p1, const Doub *p2) {
```

Euclidean distance.

```
        Doub sum = 0.;
        for (Int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
        return sqrt(sum);
    }
};
```

3.7.4 Interpolation by Kriging

Kriging is a technique named for South African mining engineer D.G. Krige. It is basically a form of linear prediction (§13.6), also known in different communities as *Gauss-Markov estimation* or *Gaussian process regression*.

Kriging can be either an interpolation method or a fitting method. The distinction between the two is whether the fitted/interpolated function goes exactly through all the input data points (interpolation), or whether it allows measurement errors to be specified and then "smooths" to get a statistically better predictor that does not

generally go through the data points (does not "honor the data"). In this section we consider only the former case, that is, interpolation. We will return to the latter case in §15.9.

At this point in the book, it is beyond our scope to derive the equations for kriging. You can turn to §13.6 to get a flavor, and look to references [9,10,11] for details. To use kriging, you must be able to estimate the mean square variation of your function $y(\mathbf{x})$ as a function of offset distance $\mathbf{r}$, a so-called *variogram*,

$$v(\mathbf{r}) \sim \tfrac{1}{2}\left\langle [y(\mathbf{x}+\mathbf{r}) - y(\mathbf{x})]^2 \right\rangle \tag{3.7.11}$$

where the average is over all $\mathbf{x}$ with fixed $\mathbf{r}$. If this seems daunting, don't worry. For interpolation, even very crude variogram estimates work fine, and we will give below a routine to estimate $v(\mathbf{r})$ from your input data points $\mathbf{x}_i$ and $y_i = y(\mathbf{x}_i)$, $i = 0, \ldots, N-1$, automatically. One usually takes $v(\mathbf{r})$ to be a function only of the magnitude $r = |\mathbf{r}|$ and writes it as $v(r)$.

Let v_{ij} denote $v(|\mathbf{x}_i - \mathbf{x}_j|)$, where i and j are input points, and let v_{*j} denote $v(|\mathbf{x}_* - \mathbf{x}_j|)$, $\mathbf{x}_*$ being a point at which we want an interpolated value $y(\mathbf{x}_*)$. Now define two vectors of length $N+1$,

$$\begin{aligned} \mathbf{Y} &= (y_0, y_1, \ldots, y_{N-1}, 0) \\ \mathbf{V}_* &= (v_{*1}, v_{*2}, \ldots, v_{*,N-1}, 1) \end{aligned} \tag{3.7.12}$$

and an $(N+1) \times (N+1)$ symmetric matrix,

$$\mathbf{V} = \begin{pmatrix} v_{00} & v_{01} & \ldots & v_{0,N-1} & 1 \\ v_{10} & v_{11} & \ldots & v_{1,N-1} & 1 \\ & & \ldots & & \ldots \\ v_{N-1,0} & v_{N-1,1} & \ldots & v_{N-1,N-1} & 1 \\ 1 & 1 & \ldots & 1 & 0 \end{pmatrix} \tag{3.7.13}$$

Then the kriging interpolation estimate $\hat{y}_* \approx y(\mathbf{x}_*)$ is given by

$$\hat{y}_* = \mathbf{V}_* \cdot \mathbf{V}^{-1} \cdot \mathbf{Y} \tag{3.7.14}$$

and its variance is given by

$$\mathrm{Var}(\hat{y}_*) = \mathbf{V}_* \cdot \mathbf{V}^{-1} \cdot \mathbf{V}_* \tag{3.7.15}$$

Notice that if we compute, once, the LU decomposition of $\mathbf{V}$, and then backsubstitute, once, to get the vector $\mathbf{V}^{-1} \cdot \mathbf{Y}$, then the individual interpolations cost only $O(N)$: Compute the vector $\mathbf{V}_*$ and take a vector dot product. On the other hand, every computation of a variance, equation (3.7.15), requires an $O(N^2)$ backsubstitution.

As an aside (if you have looked ahead to §13.6) the purpose of the extra row and column in $\mathbf{V}$, and extra last components in $\mathbf{V}_*$ and $\mathbf{Y}$, is to automatically calculate, and correct for, an appropriately weighted average of the data, and thus to make equation (3.7.14) an unbiased estimator.

Here is an implementation of equations (3.7.12) – (3.7.15). The constructor does the one-time work, while the two overloaded `interp` methods calculate either an interpolated value or else a value and a standard deviation (square root of the variance). You should leave the optional argument `err` set to the default value of `NULL` until you read §15.9.

krig.h
```
template<class T>
struct Krig {
```
Object for interpolation by kriging, using npt points in ndim dimensions. Call constructor once, then interp as many times as desired.
```
    const MatDoub &x;
    const T &vgram;
    Int ndim, npt;
    Doub lastval, lasterr;
    VecDoub y,dstar,vstar,yvi;
    MatDoub v;
    LUdcmp *vi;
```
Most recently computed value and (if computed) error.
```
    Krig(MatDoub_I &xx, VecDoub_I &yy, T &vargram, const Doub *err=NULL)
    : x(xx),vgram(vargram),npt(xx.nrows()),ndim(xx.ncols()),dstar(npt+1),
    vstar(npt+1),v(npt+1,npt+1),y(npt+1),yvi(npt+1) {
```
Constructor. The npt × ndim matrix xx inputs the data points, the vector yy the function values. vargram is the variogram function or functor. The argument err is not used for interpolation; see §15.9.
```
        Int i,j;
        for (i=0;i<npt;i++) {
            y[i] = yy[i];
            for (j=i;j<npt;j++) {
                v[i][j] = v[j][i] = vgram(rdist(&x[i][0],&x[j][0]));
            }
            v[i][npt] = v[npt][i] = 1.;
        }
        v[npt][npt] = y[npt] = 0.;
        if (err) for (i=0;i<npt;i++) v[i][i] -= SQR(err[i]);
        vi = new LUdcmp(v);
        vi->solve(y,yvi);
    }
    ~Krig() { delete vi; }
```
Fill $\mathbf{Y}$ and $\mathbf{V}$.

§15.9.
```
    Doub interp(VecDoub_I &xstar) {
```
Return an interpolated value at the point xstar.
```
        Int i;
        for (i=0;i<npt;i++) vstar[i] = vgram(rdist(&xstar[0],&x[i][0]));
        vstar[npt] = 1.;
        lastval = 0.;
        for (i=0;i<=npt;i++) lastval += yvi[i]*vstar[i];
        return lastval;
    }

    Doub interp(VecDoub_I &xstar, Doub &esterr) {
```
Return an interpolated value at the point xstar, and return its estimated error as esterr.
```
        lastval = interp(xstar);
        vi->solve(vstar,dstar);
        lasterr = 0;
        for (Int i=0;i<=npt;i++) lasterr += dstar[i]*vstar[i];
        esterr = lasterr = sqrt(MAX(0.,lasterr));
        return lastval;
    }

    Doub rdist(const Doub *x1, const Doub *x2) {
```
Utility used internally. Cartesian distance between two points.
```
        Doub d=0.;
        for (Int i=0;i<ndim;i++) d += SQR(x1[i]-x2[i]);
        return sqrt(d);
    }
};
```

The constructor argument vgram, the variogram function, can be either a func-

tion or functor (§1.3.3). For interpolation, you can use a `Powvargram` object that fits a simple model

$$v(r) = \alpha r^{\beta} \tag{3.7.16}$$

where β is considered fixed and α is fitted by unweighted least squares over all pairs of data points i and j. We'll get more sophisticated about variograms in §15.9; but for interpolation, excellent results can be obtained with this simple choice. The value of β should be in the range $1 \leq \beta < 2$. A good general choice is 1.5, but for functions with a strong linear trend, you may want to experiment with values as large as 1.99. (The value 2 gives a degenerate matrix and meaningless results.) The optional argument nug will be explained in §15.9.

krig.h

```
struct Powvargram {
```
Functor for variogram $v(r) = \alpha r^{\beta}$, where β is specified, α is fitted from the data.
```
    Doub alph, bet, nugsq;

    Powvargram(MatDoub_I &x, VecDoub_I &y, const Doub beta=1.5, const Doub nug=0.)
    : bet(beta), nugsq(nug*nug) {
```
    Constructor. The `npt` × `ndim` matrix `x` inputs the data points, the vector `y` the function values, `beta` the value of β. For interpolation, the default value of `beta` is usually adequate. For the (rare) use of `nug` see §15.9.
```
        Int i,j,k,npt=x.nrows(),ndim=x.ncols();
        Doub rb,num=0.,denom=0.;
        for (i=0;i<npt;i++) for (j=i+1;j<npt;j++) {
            rb = 0.;
            for (k=0;k<ndim;k++) rb += SQR(x[i][k]-x[j][k]);
            rb = pow(rb,0.5*beta);
            num += rb*(0.5*SQR(y[i]-y[j]) - nugsq);
            denom += SQR(rb);
        }
        alph = num/denom;
    }

    Doub operator() (const Doub r) const {return nugsq+alph*pow(r,bet);}
};
```

Sample code for interpolating on a set of data points is

```
MatDoub x(npts,ndim);
VecDoub y(npts), xstar(ndim);
...
Powvargram vgram(x,y);
Krig<Powvargram> krig(x,y,vgram);
```

followed by any number of interpolations of the form

```
ystar = krig.interp(xstar);
```

Be aware that while the interpolated values are quite insensitive to the variogram model, the estimated errors are rather sensitive to it. You should thus consider the error estimates as being order of magnitude only. Since they are also relatively expensive to compute, their value in this application is not great. They will be much more useful in §15.9, when our model includes measurement errors.

3.7.5 Curve Interpolation in Multidimensions

A different kind of interpolation, worth a brief mention here, is when you have an ordered set of N tabulated points in n dimensions that lie on a one-dimensional curve, $\mathbf{x}_0, \ldots \mathbf{x}_{N-1}$, and you want to interpolate other values along the curve. Two